Practical 14

Capstone — Integration, JUnit Testing & Packaging (Milestone 3)

Part A — Practice Programs (Hour 1)

1) Assemble and Test — layered task app with JUnit tests

Code

Task.java

```
public record Task(String title, boolean done) { }
```

TaskService.java

```
import java.util.ArrayList;
import java.util.List;

public class TaskService {
    private final List<Task> tasks = new ArrayList<>();

    public void add(String title) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Title must not be blank");
        }
        tasks.add(new Task(title.trim(), false));
    }

    public int count() { return tasks.size(); }

    public List<Task> list() { return List.copyOf(tasks); }
}
```

TaskApp.java

```
import java.util.Scanner;

public class TaskApp {
    public static void main(String[] args) {
        TaskService service = new TaskService();
        Scanner in = new Scanner(System.in);
        while (true) {
            System.out.print("(a)dd, (l)ist, (q)uit: ");
            String choice = in.nextLine().trim();
            if (choice.equals("a")) {
                System.out.print("Title: ");
                try {
                    service.add(in.nextLine());
                    System.out.println("Added. Total tasks: " + service.count());
                } catch (IllegalArgumentException e) {
                    System.out.println("Error: " + e.getMessage());
                }
            } else if (choice.equals("l")) {
                for (Task t : service.list()) System.out.println(" - " + t.title());
            } else if (choice.equals("q")) {
                System.out.println("Bye");
                break;
            }
        }
    }
}
```

TaskServiceTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class TaskServiceTest {
    @Test
    void addingIncreasesCount() {
        TaskService s = new TaskService();
        s.add("Write report");
        s.add("Submit lab");
        assertEquals(2, s.count());
    }

    @Test
    void blankTitleThrows() {
        TaskService s = new TaskService();
        assertThrows(IllegalArgumentException.class, () -> s.add("   "));
        assertEquals(0, s.count());
    }

    @Test
    void listKeepsInsertionOrder() {
        TaskService s = new TaskService();
        s.add("first");
        s.add("second");
        assertEquals("first", s.list().get(0).title());
        assertEquals("second", s.list().get(1).title());
    }
}
```

Output

```
(a)dd, (l)ist, (q)uit: a
Title: Write report
Added. Total tasks: 1
(a)dd, (l)ist, (q)uit: a
Title:
Error: Title must not be blank
(a)dd, (l)ist, (q)uit: a
Title: Submit lab
Added. Total tasks: 2
(a)dd, (l)ist, (q)uit: l
 - Write report
 - Submit lab
(a)dd, (l)ist, (q)uit: q
Bye
```

Output

```
$ java -jar junit-platform-console-standalone.jar --scan-class-path
.
+-- JUnit Platform Suite [OK]
+-- JUnit Jupiter [OK]
| '-- TaskServiceTest [OK]
|   +-- blankTitleThrows() [OK]
|   +-- addingIncreasesCount() [OK]
|   '-- listKeepsInsertionOrder() [OK]
'-- JUnit Vintage [OK]

Test run finished after 132 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         3 tests found           ]
[         0 tests skipped         ]
[         3 tests started         ]
[         0 tests aborted         ]
[         3 tests successful      ]
[         0 tests failed          ]
```

Typed input is shown after each prompt. The blank title is rejected, so the count stays 2.

2) Packages, runnable JAR and save/reload test (For Advanced Learners — home)

Code

app/Note.java

```
package app;

import java.io.Serializable;

public record Note(String text) implements Serializable { }
```

app/NoteService.java

```
package app;

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class NoteService {
    private List<Note> notes = new ArrayList<>();

    public void add(String text) { notes.add(new Note(text)); }

    public List<Note> list() { return List.copyOf(notes); }

    public void save(File file) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(file))) {
            out.writeObject(new ArrayList<>(notes));
        }
    }

    @SuppressWarnings("unchecked")
    public void load(File file) throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(file))) {
            notes = (List<Note>) in.readObject();
        }
    }
}
```

app/Main.java

```
package app;

import java.io.File;

public class Main {
    public static void main(String[] args) throws Exception {
        File file = new File("notes.dat");
        NoteService service = new NoteService();
        service.add("Buy milk");
        service.add("Finish OOP lab");
        service.save(file);
        NoteService reopened = new NoteService();
        reopened.load(file);
        System.out.println("Saved " + service.list().size() + " notes, reloaded " +
            reopened.list().size() + ":");
        reopened.list().forEach(n -> System.out.println(" - " + n.text()));
    }
}
```

test/NoteServiceTest.java

```
package app;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

import java.io.File;
import java.nio.file.Path;

import static org.junit.jupiter.api.Assertions.assertEquals;

class NoteServiceTest {
    @TempDir
    Path dir;

    @Test
    void saveAndReloadKeepsDataUnchanged() throws Exception {
        File file = dir.resolve("notes.dat").toFile();
        NoteService a = new NoteService();
        a.add("one");
        a.add("two");
        a.save(file);

        NoteService b = new NoteService();
        b.load(file);
        assertEquals(a.list(), b.list());
    }
}
```

Code

```
javac -d out app/*.java
cd out
jar --create --file notes.jar --main-class=app.Main .
java -jar notes.jar
```

Output

```
$ java -jar junit-platform-console-standalone.jar --scan-class-path
.
+-- JUnit Platform Suite [OK]
+-- JUnit Jupiter [OK]
| '-- NoteServiceTest [OK]
|   '-- saveAndReloadKeepsDataUnchanged() [OK]
'-- JUnit Vintage [OK]

Test run finished after 157 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         1 tests found           ]
[         0 tests skipped         ]
[         1 tests started         ]
[         0 tests aborted         ]
[         1 tests successful      ]
[         0 tests failed          ]
```

Output

```
$ java -jar notes.jar
Saved 2 notes, reloaded 2:
 - Buy milk
 - Finish OOP lab
```

Part B — Project Work (Hour 2): MiniBank

All parts are assembled into one tested, packaged application. BankService wires Repository, Validator and the exception classes together; the Practical 1 menu calls real operations; accounts are loaded at start-up and saved on exit; a batch runs on the thread pool over the thread-safe store; JUnit 5 tests check the rules; the final JAR is built and a README is written.

Model and util changes — Account gets an owner name; Validator gets extra checks

Code

model/Account.java

```
package model;

import exception.BankException;
import exception.DailyLimitExceededException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

import java.io.Serializable;
import java.util.Objects;

public class Account implements Transactable, InterestBearing, Serializable, Comparable<Account> {
    private static final long serialVersionUID = 1L;
    public static final long DAILY_LIMIT = 10000;

    @Id
    @MaxLength(10)
    protected String accountNumber;
    @Positive
    protected long balance;
    private double rate;
    private String ownerName;
    private transient long withdrawnToday;   // transient: NOT saved, resets to 0 after loading

    public Account(String accountNumber, long initialBalance, double rate) {
        this(accountNumber, "-", initialBalance, rate);
    }

    public Account(String accountNumber, String ownerName, long initialBalance, double rate) {
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        this.balance = initialBalance;
        this.rate = rate;
    }
```

```
// Thread-safe: only one thread at a time may run a synchronized method of this object
@Override
public synchronized void deposit(long amount) throws InvalidAmountException {
    if (amount <= 0) {
        throw new InvalidAmountException(amount);
    }
    balance += amount;
}

@Override
public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException, DailyLimitExceededException {
    if (amount <= 0) {
        throw new InvalidAmountException(amount);
    }
    if (withdrawnToday + amount > DAILY_LIMIT) {
        throw new DailyLimitExceededException(DAILY_LIMIT);
    }
    if (amount > balance) {
        throw new InsufficientFundsException(amount - balance);
    }
    balance -= amount;
    withdrawnToday += amount;
    return true;
}

public void transfer(Account to, long amount) throws BankException {
    boolean done = false;
    try {
        withdraw(amount);
        to.deposit(amount);
        done = true;
    } catch (BankException e) {
        System.out.println("Transfer failed: " + e.getMessage());
        throw e;
    } finally {
        System.out.println("Transfer attempt " + accountNumber + " -> "
                + to.getAccountNumber() + " finished, success=" + done);
    }
}

@Override
public double interestRate() {
    return rate;
}

@Override
public long getBalance() {
    return balance;
}

public long getWithdrawnToday() {
    return withdrawnToday;
}

public String getOwnerName() {
    return ownerName;
}

public String getAccountNumber() {
    return accountNumber;
}

@Override
public int compareTo(Account other) {
    return accountNumber.compareTo(other.accountNumber);   // natural order = by account number
}

@Override
public String toString() {
    return "Account[" + accountNumber + ", balance=" + balance + "]";
```

```
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (!(o instanceof Account)) {
            return false;
        }
        Account other = (Account) o;
        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

util/Validator.java

```
package util;

import java.util.regex.Pattern;

public class Validator {
    private static final Pattern MOBILE = Pattern.compile("^[6-9][0-9]{9}$");
    private static final Pattern AMOUNT = Pattern.compile("^[1-9][0-9]*$");

    public static boolean isPositive(long amount) {
        return amount > 0;
    }

    public static boolean hasSufficientBalance(long balance, long amount) {
        return balance >= amount;
    }

    public static boolean isValidMobile(String s) {
        return s != null && MOBILE.matcher(s).matches();
    }

    public static boolean isValidAmount(String s) {
        return s != null && AMOUNT.matcher(s).matches();
    }

    public static boolean isNotBlank(String s) {
        return s != null && !s.isBlank();
    }
}
```

Repository — backing map switched to ConcurrentHashMap (thread-safe store)	
Code	**service/Repository.java**

```
package service;

import model.annotation.Id;

import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class Repository<T> {
    private final Map<String, T> store = new ConcurrentHashMap<>();   // thread-safe store

    /** Finds the field marked @Id by reflection and uses its value as the key. */
    private String idOf(T entity) {
        for (Field f : entity.getClass().getDeclaredFields()) {
            if (f.isAnnotationPresent(Id.class)) {
                try {
                    f.setAccessible(true);
                    return String.valueOf(f.get(entity));
                } catch (IllegalAccessException e) {
                    throw new IllegalStateException("Cannot read @Id field", e);
                }
            }
        }
        throw new IllegalArgumentException(entity.getClass().getSimpleName() + " has no @Id field");
    }

    public T save(T entity) {
        store.put(idOf(entity), entity);
        return entity;
    }

    public T findById(String id) {
        return store.get(id);
    }
    public ArrayList<T> findAll() {
        return new ArrayList<>(store.values());
    }

    public boolean delete(String id) {
        return store.remove(id) != null;
    }

    // supplementary: number of stored items
    public int count() {
        return store.size();
    }

    @SafeVarargs
    public final void saveAll(T... items) {
        for (T item : items) {
            save(item);
        }
    }

    /** Bounded type parameter: only Number subclasses are accepted. */
    public static <N extends Number> double average(N[] values) {
        double sum = 0;
        for (N v : values) {
            sum += v.doubleValue();
        }
        return values.length == 0 ? 0 : sum / values.length;
    }
}
```

1) BankService — openAccount, deposit, withdraw, transfer, statement, listAccounts, batch, report	
Code	**service/BankService.java**

```
package service;

import exception.AccountNotFoundException;
import exception.BankException;
import exception.InvalidAmountException;
import model.Account;
import util.Validator;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class BankService {
    private final Repository<Account> repository = new Repository<>();
    private final List<String> history = Collections.synchronizedList(new ArrayList<>());
    private final AtomicInteger counter = new AtomicInteger(0);
    private final Path accountsFile;
    private final Path logsDir;
    private final Path reportFile;

    public BankService(Path dataDir) throws IOException, ClassNotFoundException {
        this.accountsFile = dataDir.resolve("accounts.dat");
        this.logsDir = dataDir.resolve("logs");
        this.reportFile = dataDir.resolve("report.txt");
        Files.createDirectories(logsDir);
        if (Files.exists(accountsFile)) {                       // load saved accounts on start-up
            for (Account a : StatePersister.load(accountsFile)) {
                repository.save(a);
                counter.accumulateAndGet(Integer.parseInt(a.getAccountNumber().substring(2)),
                    Math::max);
            }
        }
    }

    public void save() throws IOException {
        StatePersister.save(repository.findAll().toArray(new Account[0]), accountsFile);
    }

    private Account find(String accountNumber) throws AccountNotFoundException {
        Account a = repository.findById(accountNumber);
        if (a == null) {
            throw new AccountNotFoundException(accountNumber);
        }
        return a;
    }

    private void record(String line) {
        history.add(line);
        try {
            TransactionLog.append(logsDir.resolve("txn-" + LocalDate.now() + ".log"), line);
        } catch (IOException e) {
            System.out.println("Warning: could not write log: " + e.getMessage());
        }
    }

    public Account openAccount(String owner, String mobile, long opening) throws BankException {
        if (!Validator.isNotBlank(owner)) {
            throw new BankException("Owner name must not be blank");
        }
        if (!Validator.isValidMobile(mobile)) {
            throw new BankException("Invalid mobile number: " + mobile);
        }
        if (opening < 0) {
            throw new InvalidAmountException(opening);
        }
        String number = String.format("AC%04d", counter.incrementAndGet());
        Account account = new Account(number, owner, opening, 0.04);
```

```
        repository.save(account);
        history.add(number + " OPENED for " + owner);
        return account;
    }

    public void deposit(String accountNumber, long amount) throws BankException {
        Account a = find(accountNumber);
        a.deposit(amount);
        record("DEPOSIT " + accountNumber + " " + amount);
    }

    public void withdraw(String accountNumber, long amount) throws BankException {
        Account a = find(accountNumber);
        a.withdraw(amount);
        record("WITHDRAW " + accountNumber + " " + amount);
    }

    public void transfer(String from, String to, long amount) throws BankException {
        Account source = find(from);
        Account target = find(to);
        source.withdraw(amount);
        target.deposit(amount);
        record("WITHDRAW " + from + " " + amount);
        record("DEPOSIT " + to + " " + amount);
    }

    public String statement(String accountNumber) throws BankException {
        Account a = find(accountNumber);
        StringBuilder sb = new StringBuilder();
        sb.append("======== Statement ").append(accountNumber).append(" ========\n");
        sb.append("Owner   : ").append(a.getOwnerName()).append("\n");
        sb.append("Balance : ").append(a.getBalance()).append("\n");
        sb.append("Recent transactions:\n");
        synchronized (history) {
            for (String h : history) {
                if (h.contains(accountNumber)) sb.append("  ").append(h).append("\n");
            }
        }
        sb.append("====================================");
        return sb.toString();
    }

    public List<Account> listAccounts() {
        List<Account> list = repository.findAll();
        Collections.sort(list);
        return list;
    }

    // supplementary: search by owner name
    public List<Account> searchByOwner(String text) {
        List<Account> result = new ArrayList<>();
        for (Account a : listAccounts()) {
            if (a.getOwnerName().toLowerCase().contains(text.toLowerCase())) result.add(a);
        }
        return result;
    }

    /** Runs a batch on the thread pool over the thread-safe store: 20 deposits of 5 into every
        account. */
    public String runBatch() throws InterruptedException {
        TransactionProcessor processor = new TransactionProcessor();
        List<Account> accounts = listAccounts();
        for (Account a : accounts) {
            processor.register(a);
            for (int i = 0; i < 20; i++) {
                final String number = a.getAccountNumber();
                processor.submit(() -> {
                    try {
                        deposit(number, 5);
                    } catch (BankException e) {
                        System.out.println("Batch task failed: " + e.getMessage());
                    }
```

```
                });
            }
        }
        processor.stop();
        return "Batch done: " + (accounts.size() * 20) + " deposits of 5 processed on 4 threads";
    }

    public String report() throws IOException {
        return ReportGenerator.generate(logsDir, reportFile);
    }
}
```

Supplementary problem covered: searchByOwner() powers the SEARCH_OWNER menu option.

2) Menu — BankInfo record, MenuOption enum and the switch-expression menu loop in MiniBank

Code

service/BankInfo.java

```
package service;

public record BankInfo(String name, String branch) { }
```

service/MenuOption.java

```
package service;

public enum MenuOption {
    OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, STATEMENT, LIST_ACCOUNTS, SEARCH_OWNER, BATCH,
        REPORT, EXIT
}
```

service/MiniBank.java

```
package service;

import exception.BankException;
import model.Account;
import util.Validator;

import java.nio.file.Path;
import java.util.Scanner;

public class MiniBank {
    private static String ask(Scanner in, String prompt) {
        System.out.print(prompt);
        return in.nextLine().trim();
    }

    private static long askAmount(Scanner in, String prompt) throws BankException {
        String text = ask(in, prompt);
        if (!Validator.isValidAmount(text)) {
            throw new exception.InvalidAmountException(text.matches("-?\\d+") ?
                Long.parseLong(text) : 0);
        }
        return Long.parseLong(text);
    }

    private static String handle(MenuOption option, BankService bank, Scanner in) throws Exception {
        try {
            return switch (option) {
                case OPEN_ACCOUNT -> {
                    String owner = ask(in, "Owner name: ");
                    String mobile = ask(in, "Mobile: ");
                    long opening = askAmount(in, "Opening balance: ");
                    Account a = bank.openAccount(owner, mobile, opening);
                    yield "Opened " + a.getAccountNumber() + " for " + owner;
                }
                case DEPOSIT -> {
                    String no = ask(in, "Account number: ");
```

```
                    bank.deposit(no, askAmount(in, "Amount: "));
                    yield "Deposit successful";
                }
                case WITHDRAW -> {
                    String no = ask(in, "Account number: ");
                    bank.withdraw(no, askAmount(in, "Amount: "));
                    yield "Withdrawal successful";
                }
                case TRANSFER -> {
                    String from = ask(in, "From account: ");
                    String to = ask(in, "To account: ");
                    bank.transfer(from, to, askAmount(in, "Amount: "));
                    yield "Transfer successful";
                }
                case STATEMENT -> bank.statement(ask(in, "Account number: "));
                case LIST_ACCOUNTS -> {
                    StringBuilder sb = new StringBuilder();
                    for (Account a : bank.listAccounts()) {
                        sb.append(String.format("%s | %-6s | %6d%n", a.getAccountNumber(),
                            a.getOwnerName(), a.getBalance()));
                    }
                    yield sb.toString().stripTrailing();
                }
                case SEARCH_OWNER -> {
                    StringBuilder sb = new StringBuilder();
                    for (Account a : bank.searchByOwner(ask(in, "Name contains: "))) {
                        sb.append(a.getAccountNumber()).append(" |
                            ").append(a.getOwnerName()).append("\n");
                    }
                    yield sb.length() == 0 ? "No match" : sb.toString().stripTrailing();
                }
                case BATCH -> bank.runBatch();
                case REPORT -> bank.report();
                case EXIT -> "Saving accounts... Goodbye!";
            };
        } catch (BankException e) {
            return "Error: " + e.getMessage();
        }
    }

    public static void main(String[] args) throws Exception {
        BankService bank = new BankService(Path.of("data"));
        Scanner in = new Scanner(System.in);
        MenuOption[] options = MenuOption.values();
        System.out.println(new BankInfo("MiniBank", "Surat Main Branch"));

        MenuOption chosen = null;
        while (chosen != MenuOption.EXIT) {
            System.out.println("\n--- MENU ---");
            for (int i = 0; i < options.length; i++) {
                System.out.println((i + 1) + ". " + options[i]);
            }
            String line = ask(in, "Choose: ");
            int n;
            try {
                n = Integer.parseInt(line);
            } catch (NumberFormatException e) {
                n = -1;
            }
            if (n < 1 || n > options.length) {
                System.out.println("Invalid choice, enter a number from 1 to " + options.length);
                continue;
            }
            chosen = options[n - 1];
            System.out.println(handle(chosen, bank, in));
        }
        bank.save();
    }
}
```

2-3) Running the final JAR — persistence across restarts, batch and report

Output

```
$ java -jar minibank.jar      (first run - fresh data folder)
BankInfo[name=MiniBank, branch=Surat Main Branch]

--- MENU ---
1. OPEN_ACCOUNT
2. DEPOSIT
3. WITHDRAW
4. TRANSFER
5. STATEMENT
6. LIST_ACCOUNTS
7. SEARCH_OWNER
8. BATCH
9. REPORT
10. EXIT
Choose: 1
Owner name: Riya
Mobile: 9876543210
Opening balance: 5000
Opened AC0001 for Riya

Choose: 1
Owner name: Aman
Mobile: 9123456780
Opening balance: 2000
Opened AC0002 for Aman

Choose: 1
Owner name: Neha
Mobile: 12345
Opening balance: 1000
Error: Invalid mobile number: 12345

Choose: 2
Account number: AC0001
Amount: 1500
Deposit successful

Choose: 3
Account number: AC0002
Amount: 9000
Error: Insufficient funds, short by 7000

Choose: 3
Account number: AC0002
Amount: 500
Withdrawal successful

Choose: 4
From account: AC0001
To account: AC0002
Amount: 700
Transfer successful

Choose: 42
Invalid choice, enter a number from 1 to 10

Choose: 5
Account number: AC0001
======== Statement AC0001 ========
Owner   : Riya
Balance : 5800
Recent transactions:
  AC0001 OPENED for Riya
  DEPOSIT AC0001 1500
  WITHDRAW AC0001 700
====================================

Choose: 6
AC0001 | Riya   |   5800
AC0002 | Aman   |   2200

Choose: 7
```

```
Name contains: ri
AC0001 | Riya

Choose: 8
Batch done: 40 deposits of 5 processed on 4 threads

Choose: 6
AC0001 | Riya   |   5900
AC0002 | Aman   |   2300

Choose: 9
===== End-of-day report =====
File txn-2026-10-20.log | size 759 bytes | modified 2026-10-20T18:12:02.305243472Z
Total deposits    : 2400
Total withdrawals : 1200
Net change        : 1200

Choose: 10
Saving accounts... Goodbye!
```

Output

```
$ java -jar minibank.jar      (second run - accounts reloaded from data/accounts.dat)
BankInfo[name=MiniBank, branch=Surat Main Branch]

--- MENU ---
1. OPEN_ACCOUNT
2. DEPOSIT
3. WITHDRAW
4. TRANSFER
5. STATEMENT
6. LIST_ACCOUNTS
7. SEARCH_OWNER
8. BATCH
9. REPORT
10. EXIT
Choose: 1
Owner name: Karan
Mobile: 9988776655
Opening balance: 300
Opened AC0003 for Karan

Choose: 6
AC0001 | Riya   |   5900
AC0002 | Aman   |   2300
AC0003 | Karan  |    300

Choose: 9
===== End-of-day report =====
File txn-2026-10-20.log | size 759 bytes | modified 2026-10-20T18:12:02.305243472Z
Total deposits    : 2400
Total withdrawals : 1200
Net change        : 1200

Choose: 10
Saving accounts... Goodbye!
```

Output

```
$ ls -R data
data:
accounts.dat
logs
report.txt

data/logs:
txn-2026-10-20.log
```

The menu listing is shown in full once per run; repeated listings are abbreviated here to save space. Invalid number 42 is re-prompted, an invalid mobile and an overdraw are reported as friendly errors, and the second run reloads AC0001 and AC0002 from data/accounts.dat before opening AC0003.

4) JUnit 5 tests — deposit, over-limit withdrawal, concurrency, save/load

Code `test/service/BankServiceTest.java`

```
package service;

import exception.AccountNotFoundException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import model.Account;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

import java.nio.file.Path;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

import static org.junit.jupiter.api.Assertions.*;

class BankServiceTest {
    @TempDir
    Path dir;

    @Test
    void depositIncreasesBalance() throws Exception {
        BankService bank = new BankService(dir);
        Account a = bank.openAccount("Riya", "9876543210", 1000);
        bank.deposit(a.getAccountNumber(), 500);
        assertEquals(1500, a.getBalance());
    }

    @Test
    void withdrawalOverBalanceThrows() throws Exception {
        BankService bank = new BankService(dir);
        Account a = bank.openAccount("Riya", "9876543210", 1000);
        InsufficientFundsException e = assertThrows(InsufficientFundsException.class,
                () -> bank.withdraw(a.getAccountNumber(), 5000));
        assertEquals(4000, e.getShortfall());
        assertEquals(1000, a.getBalance());          // unchanged after the failure
    }

    @Test
    void invalidAmountAndUnknownAccountThrow() throws Exception {
        BankService bank = new BankService(dir);
        Account a = bank.openAccount("Riya", "9876543210", 1000);
        assertThrows(InvalidAmountException.class, () -> bank.deposit(a.getAccountNumber(), -5));
        assertThrows(AccountNotFoundException.class, () -> bank.deposit("AC9999", 10));
    }

    @Test
    void concurrentDepositsKeepBalanceCorrect() throws Exception {
        Account a = new Account("AC0001", 0, 0.04);
        ExecutorService pool = Executors.newFixedThreadPool(10);
        for (int t = 0; t < 10; t++) {
            pool.execute(() -> {
                for (int i = 0; i < 1000; i++) {
                    try { a.deposit(1); } catch (Exception e) { fail(e); }
                }
            });
        }
        pool.shutdown();
        assertTrue(pool.awaitTermination(30, TimeUnit.SECONDS));
        assertEquals(10000, a.getBalance());
    }

    @Test
    void saveAndLoadKeepsDataUnchanged() throws Exception {
        BankService bank = new BankService(dir);
        Account a = bank.openAccount("Riya", "9876543210", 2500);
        bank.deposit(a.getAccountNumber(), 700);
        bank.save();

        BankService reopened = new BankService(dir);
        assertEquals(1, reopened.listAccounts().size());
        Account loaded = reopened.listAccounts().get(0);
        assertEquals(a.getAccountNumber(), loaded.getAccountNumber());
        assertEquals("Riya", loaded.getOwnerName());
        assertEquals(3200, loaded.getBalance());
    }
}
```

Output

```
$ java -jar junit-platform-console-standalone.jar --scan-class-path
.
+-- JUnit Platform Suite [OK]
+-- JUnit Jupiter [OK]
| '-- BankServiceTest [OK]
|   +-- depositIncreasesBalance() [OK]
|   +-- saveAndLoadKeepsDataUnchanged() [OK]
|   +-- withdrawalOverBalanceThrows() [OK]
|   +-- concurrentDepositsKeepBalanceCorrect() [OK]
|   '-- invalidAmountAndUnknownAccountThrow() [OK]
'-- JUnit Vintage [OK]

Test run finished after 251 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         5 tests found           ]
[         0 tests skipped         ]
[         5 tests started         ]
[         0 tests aborted         ]
[         5 tests successful      ]
[         0 tests failed          ]
```

Supplementary problem covered: saveAndLoadKeepsDataUnchanged. The optional @Audit annotation was not implemented.

6) README.md

Code

```
# MiniBank (Practical 14 - final)

Console banking application: layered design (model / service / util / exception), file persistence,
    thread-pool batch, JUnit tests.

## Requirements
JDK 17+ (built and tested on JDK 21). JUnit 5 console jar for the tests.

## Build
    javac -d out $(find exception model util service -name "*.java")
    cd out && jar --create --file ../minibank.jar --main-class=service.MiniBank . && cd ..

## Run
    java -jar minibank.jar
Accounts are loaded from data/accounts.dat on start-up and saved when you choose EXIT.

## Test
    javac -cp junit-platform-console-standalone.jar:out -d tout test/service/BankServiceTest.java
    java -jar junit-platform-console-standalone.jar --class-path out:tout --scan-class-path tout

## Menu
OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, STATEMENT, LIST_ACCOUNTS, SEARCH_OWNER, BATCH, REPORT,
    EXIT
```

Package layout, build and tag

Code

```
minibank/
 |-- exception/
 |   |-- AccountNotFoundException.java
 |   |-- BankException.java
 |   |-- DailyLimitExceededException.java
 |   |-- InsufficientFundsException.java
 |   +-- InvalidAmountException.java
 |-- model/
 |   |-- annotation/
 |   |   |-- Id.java
 |   |   |-- MaxLength.java
 |   |   +-- Positive.java
 |   |-- Account.java
 |   |-- Customer.java
 |   |-- InterestBearing.java
 |   |-- Premium.java
 |   |-- PremiumAccount.java
 |   +-- Transactable.java
 |-- service/
 |   |-- AccountStore.java
 |   |-- BankInfo.java
 |   |-- BankService.java
 |   |-- MenuOption.java
 |   |-- MiniBank.java
 |   |-- ReportGenerator.java
 |   |-- Repository.java
 |   |-- StatePersister.java
 |   |-- TransactionLog.java
 |   |-- TransactionProcessor.java
 |   +-- WithdrawRule.java
 |-- test/
 |   +-- service/
 |       +-- BankServiceTest.java
 |-- util/
 |   |-- AnnotationValidator.java
 |   +-- Validator.java
 +-- README.md

javac -d out $(find exception model util service -name "*.java")
javac -cp junit-platform-console-standalone.jar:out -d tout test/service/BankServiceTest.java
java -jar junit-platform-console-standalone.jar --class-path out:tout --scan-class-path tout
cd out
jar --create --file ../minibank.jar --main-class=service.MiniBank .
cd ..
java -jar minibank.jar
echo "data/" >> .gitignore
git add .
git commit -m "Practical 14 - final MiniBank"
git tag lab-14
```

All five JUnit tests pass, the JAR runs the finished MiniBank, the README explains how to build and run, and data persists across restarts.

Key Questions / Analysis / Interpretation

1. What is a layered architecture (model, service, user interface), and why is it useful?

The program is split into layers with one job each: model (Account, Customer), service (BankService, Repository: the business rules) and user interface (the menu in MiniBank). A layer only talks to the one below it. The rules can then be tested without the menu (the JUnit tests call BankService directly), and one layer can be changed (for example a GUI instead of the console) without rewriting the others.

2. How do you assert in JUnit that a method throws an expected exception?

Use assertThrows(ExpectedException.class, () -> code) from org.junit.jupiter.api.Assertions. The test passes only if the lambda throws that type (or a subclass); it returns the exception object so its details can be checked, for example assertEquals(4000, e.getShortfall()) in withdrawalOverBalanceThrows.

3. What does packaging the application as a runnable JAR provide?

A JAR bundles all compiled classes into a single file, and the manifest's Main-Class entry tells Java where main() is, so the whole application starts with one command: java -jar minibank.jar. It is easy to copy, version and deliver, and does not need the source tree or a classpath to be set by hand.